

MLA-0201 Fundamental Of Machine Learning Annexure A

Annexure A

Constraint-Based Problem Solving

Question 1 (20 Marks): Reinforcement Learning for Warehouse Robot Navigation Scenario

An e-commerce company uses an autonomous warehouse robot to transport packages between storage racks and dispatch areas. The robot must reach the destination while avoiding obstacles, minimizing travel time, and conserving battery power.

Constraints

- The robot can move only in four directions (Up, Down, Left, Right).
- Obstacles cannot be crossed.
- Each movement consumes one unit of battery.
- The robot must reach the destination within 25 moves.
- Battery capacity is limited to 30 units.
- Some paths provide positive rewards, while blocked paths incur penalties.
- The environment is stochastic, and some actions may not produce the intended movement.

Task

Design a Reinforcement Learning solution by selecting an appropriate exploration strategy and applying either **Value Iteration** or **Policy Iteration** to determine the optimal navigation policy. Justify your approach based on the given constraints.

Question 2 (20 Marks): Multi-Armed Bandit for Online Advertisement Selection Scenario

A digital marketing company displays one of several online advertisements to website visitors. The objective is to maximize the click-through rate while learning customer preferences over time.

Constraints

- Five advertisements are available.
- Only one advertisement can be displayed per visitor.
- The click probability of each advertisement is initially unknown.
- The system must balance exploration and exploitation.
- The total advertising budget allows only 10,000 user interactions.
- The solution should minimize cumulative regret while maximizing total rewards.

Task

Formulate the problem as a **K-Armed Bandit** problem. Select an appropriate exploration strategy (e.g., ϵ -greedy, UCB, or Thompson Sampling) and justify how it satisfies the given constraints. Explain how the reward mechanism influences advertisement selection.

Question 3 (20 Marks): Sampling Strategy for Machine Learning Model Development Scenario

A healthcare analytics company is developing a disease prediction model using a dataset containing one million patient records. Limited computational resources prevent training on the complete dataset.

Constraints

- Training memory is limited to 8 GB RAM.
- The sample must accurately represent all patient categories.
- The prediction accuracy should remain above 95%.
- Model confidence must be at least 95%.
- The solution should minimize sampling bias and computational cost.
- The selected learning approach should satisfy sample complexity and generalization requirements.

Task

Recommend an appropriate sampling method (e.g., Simple Random Sampling, Stratified Sampling, or Monte Carlo Sampling) and justify your choice. Analyze how **sample complexity**, **VC dimension**, **Occam's Learning Principle**, and **accuracy-confidence boosting** influence the model's performance under the given constraints.

Constraint-Based Problem Solving

Question 1: Reinforcement Learning for Warehouse Robot Navigation

1. Problem Formulation

The warehouse robot can be modeled as a **Markov Decision Process (MDP)**.

- **State (S):** Robot's current position, remaining battery, and number of moves used.
- **Actions (A):** Up, Down, Left, Right.
- **Reward (R):**
 - Positive reward for moving toward the destination.
 - Large positive reward for reaching the destination.
 - Negative reward for hitting/attempting blocked paths.
 - Small negative reward for every movement to encourage shorter paths and battery conservation.
- **Transition:** The environment is stochastic, so an action may sometimes result in an unintended movement.
- **Goal:** Reach the destination within **25 moves** while using at most **30 battery units**.

2. State Representation

A state can be represented as:

$$S = (x, y, b, m)$$

where:

- x, y = robot position
- b = remaining battery
- m = number of moves already used

state is valid only when:

$$m \leq 25$$

and

$$b \geq 0$$

Since each movement consumes one battery unit, the robot cannot make more than 30 movements based on battery capacity. However, the **25-move constraint is stricter**.

3. Exploration Strategy

I would select **ϵ -greedy exploration**.

With probability ϵ , the robot selects a random action:

$$a = \text{random}$$

With probability $1 - \epsilon$, it selects the action having the highest estimated value:

$$a = \arg \max_a Q(s, a)$$

For example:

- Initially, $\epsilon = 0.2$, allowing exploration.
- Gradually reduce ϵ to around 0.05 as the robot learns.

Why ϵ -Greedy?

It is suitable because:

1. The robot needs to **explore different paths**.
2. It can discover paths with positive rewards.

3. It avoids repeatedly choosing a poor path.
4. Reduced ϵ later allows **exploitation of the best path**.
5. It is simple and computationally efficient.

4. Value Iteration

I would use **Value Iteration** because the warehouse environment can be modeled with known transition probabilities and rewards.

The Bellman optimality equation is:

$$V(s) = \max_a [R(s, a) + \gamma \sum_{s'} P(s', | sa) V(s')]$$

where:

- $V(s)$ = value of state s
- $R(s, a)$ = reward for action
- γ = discount factor
- $P(s', | sa)$ = probability of reaching next state s'

The optimal policy is:

$$\pi(s) = \arg \max_a [R(s, a) + \gamma \sum_{s'} P(s', | sa) V(s')]$$

5. Constraint Handling

Constraint

RL Solution

Four directions only

Action space = Up, Down, Left, Right

Obstacles cannot be crossed

Assign very large negative reward to obstacle transitions

One battery per movement

Decrease battery by 1 after every action

Maximum 25 moves

Treat states after 25 moves as failure/terminal

states Battery capacity 30

Prevent actions when battery is exhausted

Positive paths

Assign positive rewards

Blocked paths

Assign negative rewards

Stochastic environment

Use transition probabilities

6. Example Reward Structure

A possible reward system is:

- Reaching destination: **+100**
- Moving closer to destination: **+5**
- Normal movement: **-1**
- Moving toward an obstacle: **-20**
- Exceeding 25 moves: **-50**
- Battery exhaustion before destination: **-50**

This encourages the robot to find a **short, safe and energy-efficient route**.

7. Conclusion

ϵ -greedy + Value Iteration is an appropriate solution. ϵ -greedy provides exploration and exploitation, while Value Iteration calculates the optimal value of each state considering stochastic movements. The reward function can simultaneously encourage **short travel time**,

obstacle avoidance, and battery conservation. The 25-move and 30-unit battery constraints can be directly incorporated into the state and terminal conditions.

Question 2: Multi-Armed Bandit for Online Advertisement Selection

1. Problem Formulation

This problem can be modeled as a **5-Armed Bandit problem**.

There are five advertisements:

$$A = \{A_1, A_2, A_3, A_4, A_5\}$$

For every website visitor, the system selects exactly **one advertisement**.

The click probability of each advertisement is initially unknown.

2. Bandit Components

- **Arms:** Five advertisements.
- **Action:** Select one advertisement.
- **Reward:** 1 if the visitor clicks and 0 otherwise.
- **No click:** Reward = 0.
- **Number of interactions:** 10,000.
- **Objective:** Maximize total clicks and minimize cumulative regret. The expected reward for advertisement i is:

$$E[R_i] = p_i$$

where p_i is the unknown click probability.

3. Recommended Strategy: UCB

I would select the **Upper Confidence Bound (UCB)** algorithm.

The UCB value for advertisement i is:

$$UCB_i = \bar{x}_i + \sqrt{\frac{2 \ln t}{n_i}}$$

where:

- $\bar{x}_i$ = observed average reward of advertisement i
- t = total number of interactions
- n_i = number of times advertisement i has been selected.

The system chooses:

$$i^* = \arg \max_i UCB_i$$

4. How UCB Balances Exploration and Exploitation

The UCB equation contains two parts:

$$\underbrace{\bar{x}_i}_{\text{Exploitation}} + \underbrace{\sqrt{\frac{2 \ln t}{n_i}}}_{\text{Exploration}}$$

Exploitation: Advertisements with high click rates receive higher values.

Exploration: Advertisements selected fewer times receive a larger uncertainty bonus.

Therefore, UCB automatically balances exploration and exploitation without requiring a manually selected ϵ .

5. Initial Stage

Initially, each advertisement should be displayed at least once:

Advertisement Initial clicks Initial observations

A1	Unknown	1+
A2	Unknown	1+
A3	Unknown	1+
A4	Unknown	1+
A5	Unknown	1+

After collecting initial results, UCB determines which advertisements should receive more traffic.

6. Reward Mechanism

The reward can be defined as:

$$R = \begin{cases} 1 & \text{if advertisement is clicked} \\ 0 & \text{otherwise} \end{cases}$$

For example, if advertisement A3 receives 100 impressions and gets 20 clicks:

$$\bar{x}_3 = \frac{20}{100} = 0.20$$

Its estimated CTR is **20%**.

An advertisement with a high CTR generally receives more selections, but advertisements with limited observations still get opportunities for exploration.

7. Cumulative Regret

Regret measures the loss caused by not always selecting the best advertisement.

If the best advertisement has click probability p^* , cumulative regret after T interactions can be expressed as:

$$Regret(T) = Tp^* - \sum_{t=1}^T R_t$$

The goal is to keep this value as small as possible.

8. Why UCB is Suitable

UCB is appropriate because:

1. There are only **five arms**.
2. The click probabilities are initially unknown.
3. It automatically balances exploration and exploitation.
4. It does not require a manually chosen ϵ .
5. It reduces unnecessary exploration as more data becomes available.
6. It works well under a fixed **10,000-interaction budget**.
7. It attempts to minimize cumulative regret.

9. Conclusion

The online advertising problem can be modeled as a **5-Armed Bandit**, where each advertisement is an arm and a click is the reward. **UCB** is a suitable strategy because it explores advertisements with high uncertainty while exploiting advertisements that have

demonstrated high CTR. This helps maximize total clicks within the limited 10,000-interaction budget.

Question 3: Sampling Strategy for Machine Learning Model Development

1. Problem Formulation

The company has:

1,000,000

patient records but cannot use the entire dataset because of the **8 GB RAM limitation**.

The sample must:

- Represent all patient categories.
- Minimize sampling bias.
- Maintain high prediction accuracy.
- Provide at least 95% confidence.
- Reduce computational cost.

2. Recommended Method: Stratified Sampling

I recommend **Stratified Random Sampling**.

In this method, the complete dataset is divided into meaningful groups called **strata**.

For example:

- Disease-positive patients
- Disease-negative patients

Additional strata could be created based on important categories such as age groups, gender, or geographic groups when appropriate.

A random sample is then selected from each stratum.

3. Why Stratified Sampling?

Simple Random Sampling may accidentally select too few examples from a minority disease category.

For example, suppose:

- 95% = healthy
- 5% = disease-positive

A random sample may not adequately represent the 5% disease group.

Stratified sampling ensures that important categories are represented.

4. Sampling Process

Suppose we select **100,000 records** instead of 1 million.

If the original dataset contains:

- 950,000 healthy records
- 50,000 disease records

A proportional sample can contain approximately:

- 95,000 healthy records
- 5,000 disease records

Therefore, the sample maintains the original distribution while reducing computational requirements.

5. Sample Complexity

Sample complexity refers to the amount of training data required for a model to learn a reliable relationship between inputs and outputs.

Generally:

- More complex models require more data.
- Higher accuracy requirements require more data.
- Higher confidence requires more data.

A simplified relationship is:

$$m \propto \frac{1}{\epsilon^2}$$

where:

- m = required sample size
- ϵ = desired error tolerance.

Therefore, the company should not select an extremely small sample. The sample should be large enough to provide reliable generalization while remaining within the 8 GB memory constraint.

6. VC Dimension

VC Dimension measures the capacity or complexity of a learning model.

A model with high VC dimension can represent more complex patterns but generally requires more training examples to generalize reliably.

The relationship can be represented conceptually as:

$$m \geq O\left(\frac{VC + \log(1/\delta)}{\epsilon^2}\right)$$

where:

- VC = VC dimension
- ϵ = acceptable error
- δ = confidence error.

For example, if confidence is 95%:

$$1 - \delta = 0.95$$

so:

$$\delta = 0.05$$

A suitable sample size should therefore account for both **model complexity and desired confidence**.

7. Occam's Learning Principle

Occam's Learning Principle states that, when possible, a simpler model is preferred over an unnecessarily complex model.

A simpler model:

- Requires fewer training examples.
- Is computationally cheaper.
- Is less likely to overfit.
- Can generalize better when appropriate.

Therefore, under the 8 GB RAM limitation, the company should avoid unnecessarily complex models and select a model whose complexity matches the available sample size.

8. Accuracy-Confidence Boosting

The target is:

$$Accuracy > 95\%$$

and:

$$\text{Confidence} \geq 95\%$$

To improve accuracy and confidence:

1. Use stratified sampling.
2. Ensure minority categories are sufficiently represented.
3. Increase sample size if validation performance is unstable.
4. Use cross-validation.
5. Remove duplicate or poor-quality records.
6. Evaluate performance on a separate test set.
7. Use appropriate model complexity.

Importantly, **95% confidence does not automatically mean 95% prediction accuracy**. Confidence is related to statistical reliability, while accuracy measures the correctness of predictions.

9. Comparison of Sampling Methods

Method	Advantages	Disadvantages	Suitability
Simple Random Sampling		Simple and easy May underrepresent minority groups	Moderate
Stratified Sampling		Represents important categories	Requires identifying strata
			Best
Monte Carlo Sampling	Useful for repeated random simulations	Less directly suited to ensuring category representation	Lower

10. Computational Benefit

Instead of training on:

1,000,000

records, the company can train on a carefully selected subset such as **100,000–200,000 records**, depending on the model, feature size, and validation results.

This reduces:

- RAM usage
- Training time
- CPU/GPU requirements
- Data-processing cost

while retaining representative information.

11. Generalization

The final sample should be evaluated against the original population distribution.

The workflow can be:

1,000,000 records → Identify strata → Stratified sampling → Training dataset → Model training → Validation → Test → Accuracy & confidence evaluation

If validation accuracy is below 95%, the sample size or model should be adjusted.

12. Conclusion

Stratified Random Sampling is the best choice because the dataset contains one million patient records and the sample must accurately represent all patient categories. It reduces sampling bias while requiring substantially fewer computational resources. Sample

complexity and VC dimension help determine whether the selected sample is large enough for reliable generalization, while Occam's principle encourages an appropriately simple model. Cross-validation and sufficient sample size can help achieve the required **95% accuracy and 95% confidence**.